Actividad regresión logística

Dr. Hugo García Tecocoatzi

Instrucciones:

Vel el ejemplo y realiza la actividad final.

In [6]:

```
# Bibliotecas necesarias
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_curve, roc_auc_score, accuracy_score)

import statsmodels.api as sm
from scipy import stats

%matplotlib inline
sns.set_style('whitegrid')
```

In [7]:

```
# Dataset: Breast Cancer Wisconsin (clasificación binaria: maligno vs benigno)
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='Diagnostico')  # 0 = maligno, 1 = benigno

df = pd.concat([X, y], axis=1)

print(f"Forma del dataset: {df.shape}")
print(f"Clases: {dict(y.value_counts())}")
print(f"Proporción de benignos: {y.mean():.3f}")
df.head()
```

```
Forma del dataset: (569, 31)
Clases: {1: np.int64(357), 0: np.int64(212)}
Proporción de benignos: 0.627
```

Out[7]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...

5 rows × 31 columns

In [8]:

```
# Para simplificar el ejercicio, usamos 4 variables representativas
vars_modelo = ['mean radius', 'mean texture', 'mean perimeter', 'mean area']

print("Estadísticas descriptivas por clase:\n")
print(df.groupby('Diagnostico')[vars_modelo].agg(['mean', 'std']).round(3))
```

```
Estadísticas descriptivas por clase:

            mean radius        mean texture        mean perimeter          \
                   mean    std         mean    std            mean     std
Diagnostico
0                17.463  3.204       21.605  3.779         115.365  21.855
1                12.147  1.781       17.915  3.995          78.075  11.807

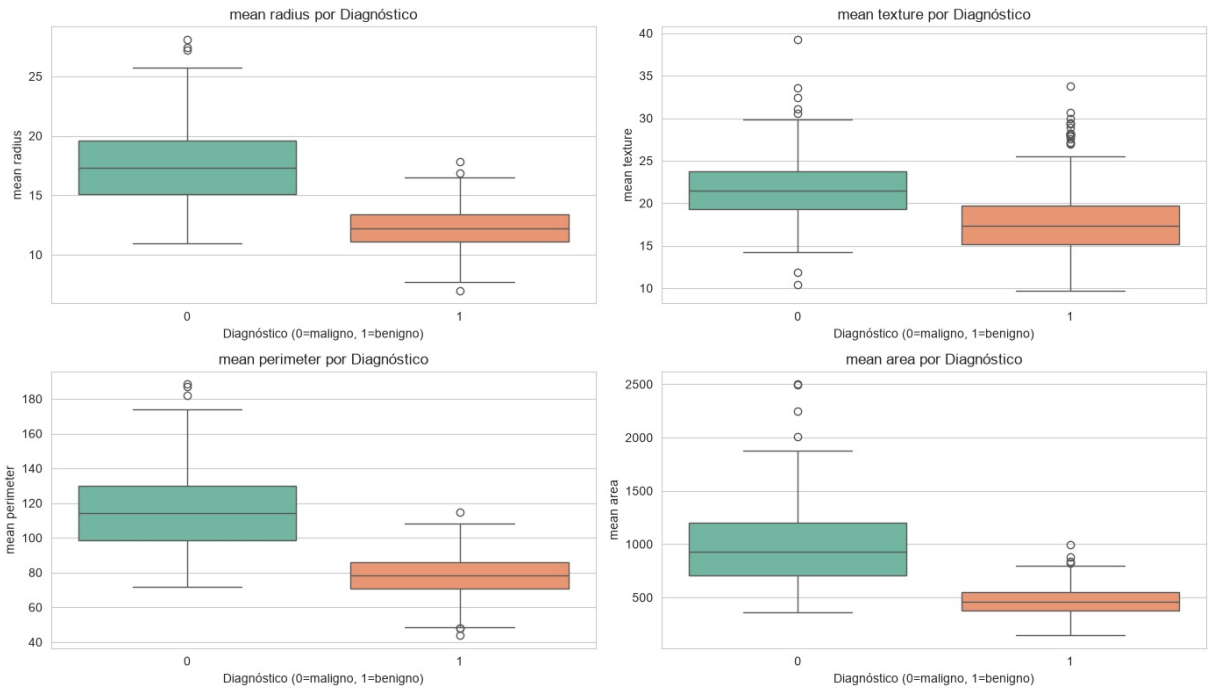
            mean area
                 mean      std
Diagnostico
0              978.376  367.938
1              462.790  134.287
```

In [9]:

```
# Boxplots por diagnóstico
fig, axes = plt.subplots(2, 2, figsize=(14, 8))
axes = axes.flatten()

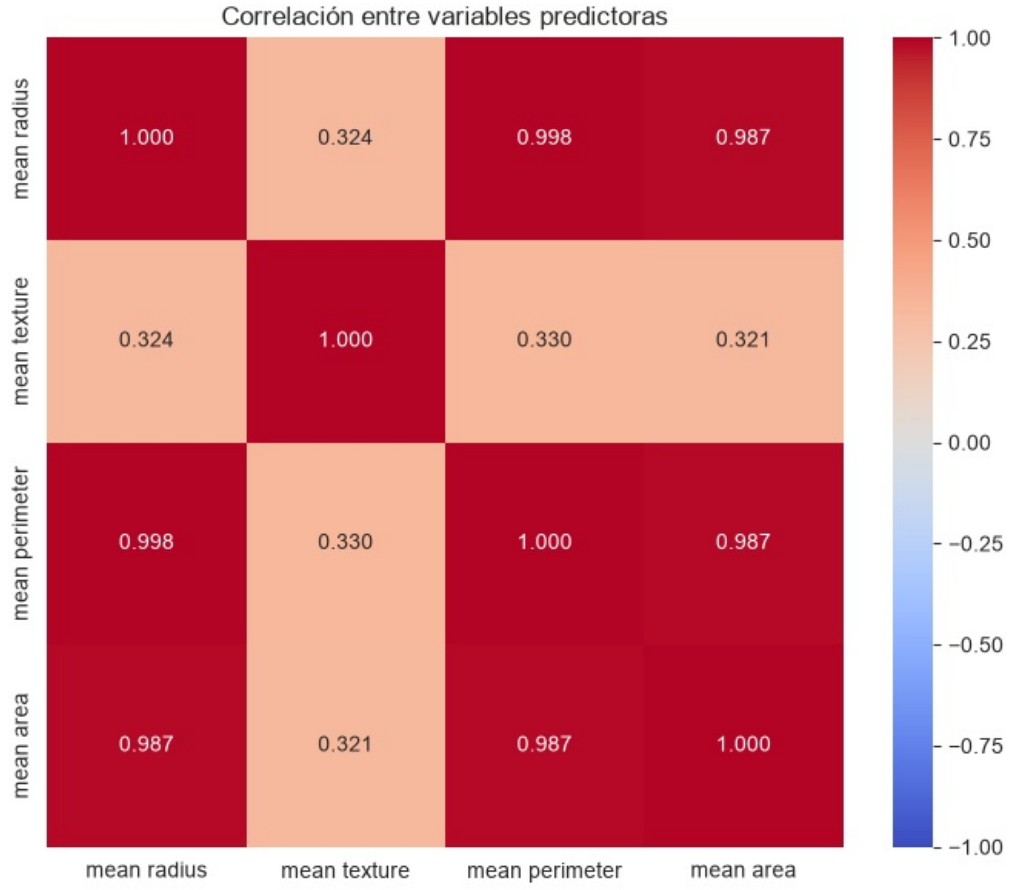
for i, var in enumerate(vars_modelo):
    sns.boxplot(x='Diagnostico', y=var, data=df, ax=axes[i], palette='Set2')
    axes[i].set_title(f'{var} por Diagnóstico')
    axes[i].set_xlabel('Diagnóstico (0=maligno, 1=benigno)')

plt.tight_layout()
plt.show()
```



In [10]:

```
# Correlación entre variables
plt.figure(figsize=(7, 6))
sns.heatmap(df[vars_modelo].corr(), annot=True, cmap='coolwarm',
            center=0, vmin=-1, vmax=1, fmt='.3f')
plt.title('Correlación entre variables predictoras')
plt.tight_layout()
plt.show()
```



In [11]:

```
# Ajustar regresión logística con statsmodels (da p-valores y OR)
X_sm = sm.add_constant(df[vars_modelo])
y_sm = df['Diagnostico']

modelo_logit = sm.Logit(y_sm, X_sm).fit(disp=False)
print(modelo_logit.summary())
```

```
                           Logit Regression Results
==============================================================================
Dep. Variable:            Diagnostico   No. Observations:                  569
Model:                          Logit   Df Residuals:                      564
Method:                           MLE   Df Model:                            4
Date:                Thu, 15 Oct 2026   Pseudo R-squ.:                  0.7187
Time:                        11:35:57   Log-Likelihood:                -105.68
converged:                       True   LL-Null:                       -375.72
Covariance Type:            nonrobust   LLR p-value:                1.437e-115
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -1.7729      6.870     -0.258      0.796     -15.238      11.692
mean radius    9.4287      1.640      5.751      0.000       6.215      12.642
mean texture  -0.2376      0.046     -5.162      0.000      -0.328      -0.147
mean perimeter -1.1507     0.164     -7.001      0.000      -1.473      -0.829
mean area     -0.0328      0.012     -2.771      0.006      -0.056      -0.010
==============================================================================

Possibly complete quasi-separation: A fraction 0.11 of observations can be
perfectly predicted. This might indicate that there is complete
quasi-separation. In this case some parameters will not be identified.
```

In [12]:

```
# Odds Ratios con intervalos de confianza al 95%
odds_ratios = pd.DataFrame({
    'Coeficiente': modelo_logit.params,
    'OR': np.exp(modelo_logit.params),
    'IC 2.5%': np.exp(modelo_logit.conf_int()[0]),
    'IC 97.5%': np.exp(modelo_logit.conf_int()[1]),
    'p-valor': modelo_logit.pvalues
}).round(4)

print("=== ODDS RATIOS ===\n")
print(odds_ratios)
```

```
=== ODDS RATIOS ===

               Coeficiente          OR   IC 2.5%     IC 97.5%  p-valor
const              -1.7729      0.1698    0.0000  119645.3293   0.7964
mean radius         9.4287  12440.8159  500.2941  309365.8457   0.0000
mean texture       -0.2376      0.7885    0.7205       0.8630   0.0000
mean perimeter     -1.1507      0.3164    0.2293       0.4367   0.0000
mean area          -0.0328      0.9678    0.9456       0.9905   0.0056
```

In [13]:

```
# Predicciones de probabilidad
y_pred_prob = modelo_logit.predict(X_sm)
y_pred = (y_pred_prob > 0.5).astype(int)

# Matriz de confusión
print("Matriz de confusión:\n")
print(confusion_matrix(y_sm, y_pred))

print("\nReporte de clasificación:\n")
print(classification_report(y_sm, y_pred, target_names=['maligno', 'benigno']))

print(f"Accuracy: {accuracy_score(y_sm, y_pred):.4f}")
```

```
Matriz de confusión:

[[183  29]
 [ 17 340]]

Reporte de clasificación:

              precision    recall  f1-score   support

     maligno       0.92      0.86      0.89       212
     benigno       0.92      0.95      0.94       357

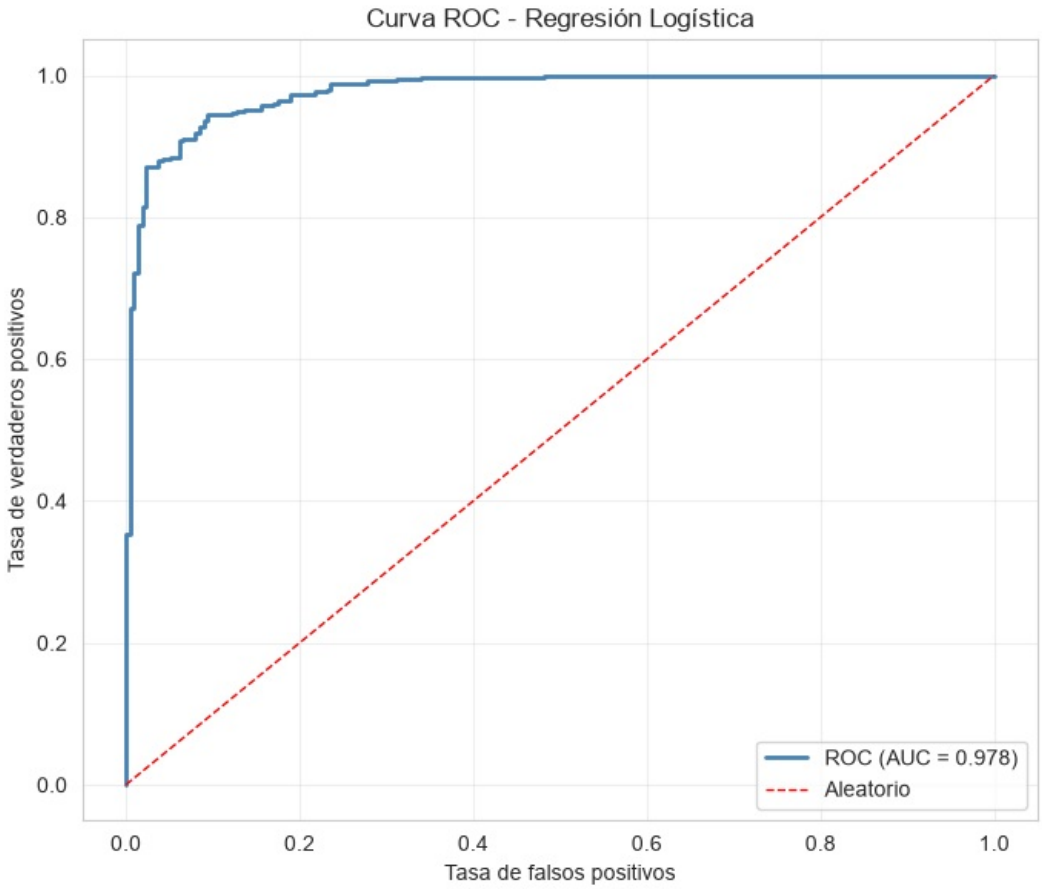
    accuracy                           0.92       569
   macro avg       0.92      0.91      0.91       569
weighted avg       0.92      0.92      0.92       569

Accuracy: 0.9192
```

In [14]:

```
# Curva ROC
fpr, tpr, _ = roc_curve(y_sm, y_pred_prob)
auc = roc_auc_score(y_sm, y_pred_prob)

plt.figure(figsize=(7, 6))
plt.plot(fpr, tpr, color='steelblue', lw=2, label=f'ROC (AUC = {auc:.3f})')
plt.plot([0, 1], [0, 1], 'r--', lw=1, label='Aleatorio')
plt.xlabel('Tasa de falsos positivos')
plt.ylabel('Tasa de verdaderos positivos')
plt.title('Curva ROC - Regresión Logística')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```


Curva ROC - Regresión Logística
Tasa de verdaderos positivos
Tasa de falsos positivos
0.0
0.2
0.4
0.6
0.8
1.0
ROC (AUC = 0.978)
Aleatorio

In [20]:

```
# Implementación de la prueba de Hosmer-Lemeshow
def hosmer_lemeshow(y_true, y_prob, g=10):
    """
    Prueba de bondad de ajuste de Hosmer-Lemeshow.
    y_true: valores observados (0/1)
    y_prob: probabilidades predichas
    g: número de grupos (por defecto 10)
    """
    # Crear DataFrame auxiliar
    df_hl = pd.DataFrame({'y': y_true, 'p': y_prob})
    df_hl['grupo'] = pd.qcut(df_hl['p'], q=g, duplicates='drop')

    # Calcular observados y esperados por grupo
    hl_tabla = df_hl.groupby('grupo').agg(
        n=('y', 'size'),
        observados=('y', 'sum'),
        esperados=('p', 'sum'),
        p_medio=('p', 'mean')
    )
    hl_tabla['obs_no'] = hl_tabla['n'] - hl_tabla['observados']
    hl_tabla['esp_no'] = hl_tabla['n'] - hl_tabla['esperados']

    # Estadístico HL
    HL = (((hl_tabla['observados'] - hl_tabla['esperados'])**2) /
      (hl_tabla['n'] * hl_tabla['p_medio'] * (1 - hl_tabla['p_medio']))).sum()

    gl = len(hl_tabla) - 2
    p_valor = 1 - stats.chi2.cdf(HL, gl)

    return HL, gl, p_valor, hl_tabla

HL, gl, p_valor, tabla = hosmer_lemeshow(y_sm, y_pred_prob, g=10)

print(f"Estadístico HL: {HL:.4f}")
print(f"Grados de libertad: {gl}")
print(f"p-valor: {p_valor:.4f}\n")

if p_valor > 0.05:
    print("✅ No se rechaza H0: el modelo tiene buen ajuste")
else:
    print("⚠️ Se rechaza H0: posible falta de ajuste")

print("\nTabla por grupos:")
print(tabla.round(3))
```

```
Estadístico HL: 17.1959
Grados de libertad: 8
p-valor: 0.0281

⚠️ Se rechaza H0: posible falta de ajuste

Tabla por grupos:
                     n  observados  esperados  p_medio  obs_no  esp_no
grupo
(-0.001, 3.74e-05]   57           0      0.000    0.000      57  57.000
(3.74e-05, 0.00606]  57           1      0.085    0.001      56  56.915
(0.00606, 0.25]      57           7      4.388    0.077      50  52.612
(0.25, 0.703]        57          24     28.073    0.493      33  28.927
(0.703, 0.904]       57          44     47.261    0.829      13   9.739
(0.904, 0.957]       56          54     52.441    0.936       2   3.559
(0.957, 0.979]       57          57     55.283    0.970       0   1.717
(0.979, 0.989]       57          56     56.132    0.985       1   0.868
(0.989, 0.994]       57          57     56.515    0.991       0   0.485
(0.994, 0.999]       57          57     56.821    0.997       0   0.179
```

In [16]:

```
# Pseudo R²
ll_modelo = modelo_logit.llf        # log-verosimilitud del modelo completo
ll_nulo = modelo_logit.llnull       # log-verosimilitud del modelo nulo
n = len(y_sm)

# McFadden
r2_mcfadden = 1 - (ll_modelo / ll_nulo)

# Cox-Snell
r2_coxsnell = 1 - np.exp((2 / n) * (ll_nulo - ll_modelo))

# Nagelkerke (ajusta Cox-Snell para que el máximo sea 1)
r2_nagelkerke = r2_coxsnell / (1 - np.exp((2 / n) * ll_nulo))

print("=== PSEUDO R² ===\n")
print(f"Log-verosimilitud modelo completo: {ll_modelo:.4f}")
print(f"Log-verosimilitud modelo nulo:     {ll_nulo:.4f}\n")
print(f"McFadden:    {r2_mcfadden:.4f}")
print(f"Cox-Snell:   {r2_coxsnell:.4f}")
print(f"Nagelkerke:  {r2_nagelkerke:.4f}")
```

```
=== PSEUDO R² ===

Log-verosimilitud modelo completo: -105.6834
Log-verosimilitud modelo nulo:     -375.7200

McFadden:    0.7187
Cox-Snell:   0.6129
Nagelkerke:  0.8362
```

In [17]:

```
# Comparación con sklearn
X_train, X_test, y_train, y_test = train_test_split(
    df[vars_modelo], df['Diagnostico'], test_size=0.3,
    random_state=42, stratify=df['Diagnostico']
)

# HL_w, gl_w, p_w, _ = hosmer_lemeshow(y_w_sm, y_prob_

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

modelo_sk = LogisticRegression(max_iter=1000)
modelo_sk.fit(X_train_sc, y_train)

y_pred_sk = modelo_sk.predict(X_test_sc)
y_prob_sk = modelo_sk.predict_proba(X_test_sc)[:, 1]

print("Accuracy en test: {:.4f}".format(accuracy_score(y_test, y_pred_sk)))
print("AUC en test:      {:.4f}".format(roc_auc_score(y_test, y_prob_sk)))
print("\nReporte de clasificación (test):\n")
print(classification_report(y_test, y_pred_sk, target_names=['maligno', 'benigno']))
```

```
Accuracy en test: 0.8889
AUC en test:      0.9647

Reporte de clasificación (test):

              precision    recall  f1-score   support

     maligno       0.87      0.83      0.85        64
     benigno       0.90      0.93      0.91       107

    accuracy                           0.89       171
   macro avg       0.88      0.88      0.88       171
weighted avg       0.89      0.89      0.89       171
```

In [18]:

```
# ACTIVIDAD: Replicar todo el análisis con el dataset de Vinos
# El dataset de vinos tiene 3 clases. Para hacer regresión logística binaria,
# debemos convertir el problema a dos clases. Por ejemplo:
#   - clase 0 (vino tinto A) vs. clases 1 y 2 (resto)
# O bien usar solo dos clases filtrando una.

print("ACTIVIDAD: Regresión Logística con el dataset de Vinos\n")
print("Instrucciones:")
print("1. Cargar el dataset con load_wine()")
print("2. Convertir el problema a binario (por ejemplo, clase 0 vs. resto)")
print("3. Seleccionar 3 o 4 variables predictoras")
print("4. Explorar los datos: estadísticas, boxplots, correlación")
print("5. Ajustar el modelo con statsmodels")
print("6. Interpretar los Odds Ratios")
print("7. Evaluar con matriz de confusión, ROC, Hosmer-Lemeshow y pseudo R²")
print("8. Comparar con sklearn")
```

```
ACTIVIDAD: Regresión Logística con el dataset de Vinos

Instrucciones:
1. Cargar el dataset con load_wine()
2. Convertir el problema a binario (por ejemplo, clase 0 vs. resto)
3. Seleccionar 3 o 4 variables predictoras
4. Explorar los datos: estadísticas, boxplots, correlación
5. Ajustar el modelo con statsmodels
6. Interpretar los Odds Ratios
7. Evaluar con matriz de confusión, ROC, Hosmer-Lemeshow y pseudo R²
8. Comparar con sklearn
```

In [21]:

```
# ============================================================
# PLANTILLA PARA LA ACTIVIDAD CON EL DATASET DE VINOS
# Completa cada sección
# ============================================================

# 1. Cargar datos
from sklearn.datasets import load_wine
wine = load_wine()
X_w = pd.DataFrame(wine.data, columns=wine.feature_names)
y_w = pd.Series(wine.target, name='Clase')
df_w = pd.concat([X_w, y_w], axis=1)


# 2. Convertir a binario
df_w['Clase_bin'] = (df_w['Clase'] == 0).astype(int)


# 3. Seleccionar variables (elige 3 o 4)
vars_w = ['alcohol', 'malic_acid', 'ash', 'total_phenols']

# 4. Visualización
print(df_w.groupby('Clase_bin')[vars_w].agg(['mean','std']).round(3))

fig, axes = plt.subplots(2, 2, figsize=(14, 8))
for i, var in enumerate(vars_w):
    sns.boxplot(x='Clase_bin', y=var, data=df_w, ax=axes.flatten()[i], palette='Set2')
    axes.flatten()[i].set_title(f'{var} por clase')
plt.tight_layout(); plt.show()

plt.figure(figsize=(7,6))
sns.heatmap(df_w[vars_w].corr(), annot=True, cmap='coolwarm', center=0, vmin=-1, vmax=1)
plt.show()


# 5. Ajustar modelo con statsmodels
X_w_sm = sm.add_constant(df_w[vars_w])
y_w_sm = df_w['Clase_bin']
modelo_w = sm.Logit(y_w_sm, X_w_sm).fit(disp=False)
print(modelo_w.summary())
```

```
print(modelo_w.summary())


# 6. Odds Ratios
or_w = pd.DataFrame({
    'Coef': modelo_w.params,
    'OR': np.exp(modelo_w.params),
    'IC 2.5%': np.exp(modelo_w.conf_int()[0]),
    'IC 97.5%': np.exp(modelo_w.conf_int()[1]),
    'p': modelo_w.pvalues
 }).round(4)
print("ODDS RATIOS \n")
print(or_w)

# 7. Evaluación
y_prob_w = modelo_w.predict(X_w_sm)
y_pred_w = (y_prob_w > 0.5).astype(int)

print("Matriz de confusión:\n")
print(confusion_matrix(y_w_sm, y_pred_w))
print("\nReporte de clasificación:\n")
print(classification_report(y_w_sm, y_pred_w, target_names=['resto', 'clase 0']))
print(f"Accuracy: {accuracy_score(y_w_sm, y_pred_w):.4f}")

    #Curva ROC
fpr_w, tpr_w, _ = roc_curve(y_w_sm, y_prob_w)
auc_w = roc_auc_score(y_w_sm, y_prob_w)
plt.figure(figsize=(7, 6))
plt.plot(fpr_w, tpr_w, color='steelblue', lw=2, label=f'ROC (AUC = {auc_w:.3f})')
plt.plot([0, 1], [0, 1], 'r--', lw=1, label='Aleatorio')
plt.xlabel('Tasa de falsos positivos')
plt.ylabel('Tasa de verdaderos positivos')
plt.title('Curva ROC - Vinos')
plt.legend(); plt.grid(True, alpha=0.3); plt.tight_layout(); plt.show()


ll_mod_w = modelo_w.llf
ll_nul_w = modelo_w.llnull
n_w = len(y_w_sm)
r2_mcf_w = 1 - (ll_mod_w / ll_nul_w)
print(f"McFadden R²: {r2_mcf_w:.4f}")


    #Hosmer-Lemeshow (descomenta tu línea y agrega el print)
HL_w, gl_w, p_w, tabla_w = hosmer_lemeshow(y_w_sm, y_prob_w, g=10)
print(f"Estadístico HL: {HL_w:.4f} | gl: {gl_w} | p-valor: {p_w:.4f}")
print(tabla_w.round(3))

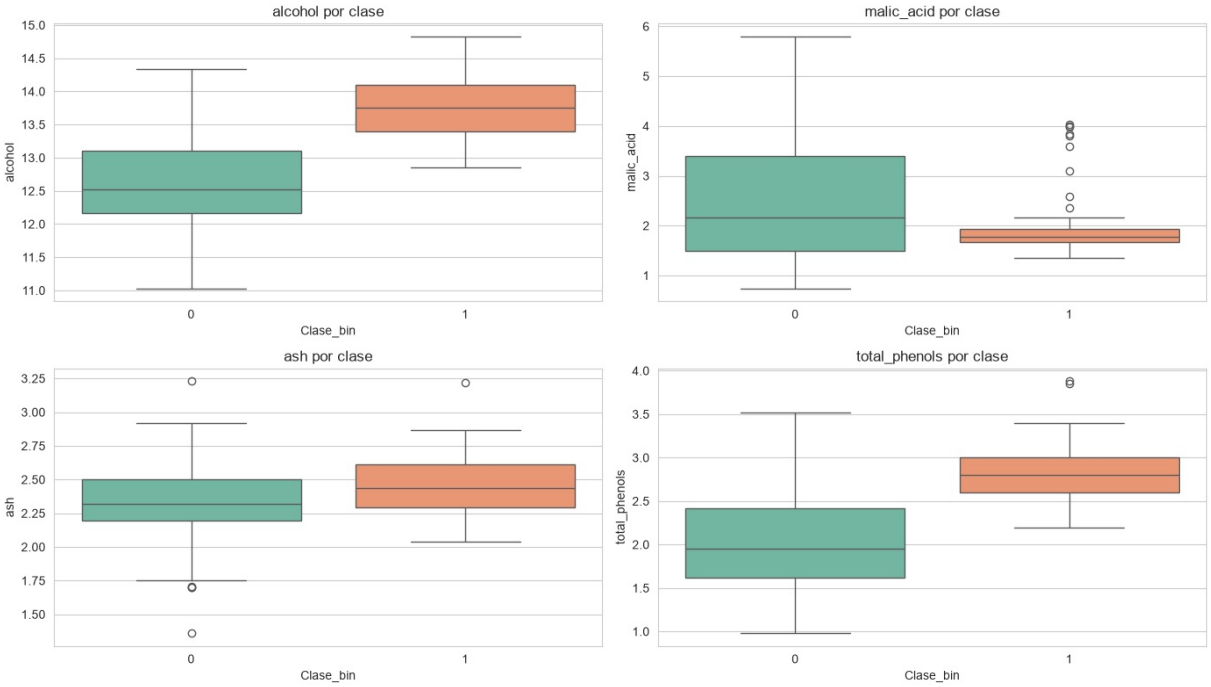
r2_cs_w = 1 - np.exp((2 / n_w) * (ll_nul_w - ll_mod_w))
r2_nag_w = r2_cs_w / (1 - np.exp((2 / n_w) * ll_nul_w))
print(f"McFadden:   {r2_mcf_w:.4f}")
print(f"Cox-Snell:  {r2_cs_w:.4f}")
print(f"Nagelkerke: {r2_nag_w:.4f}")

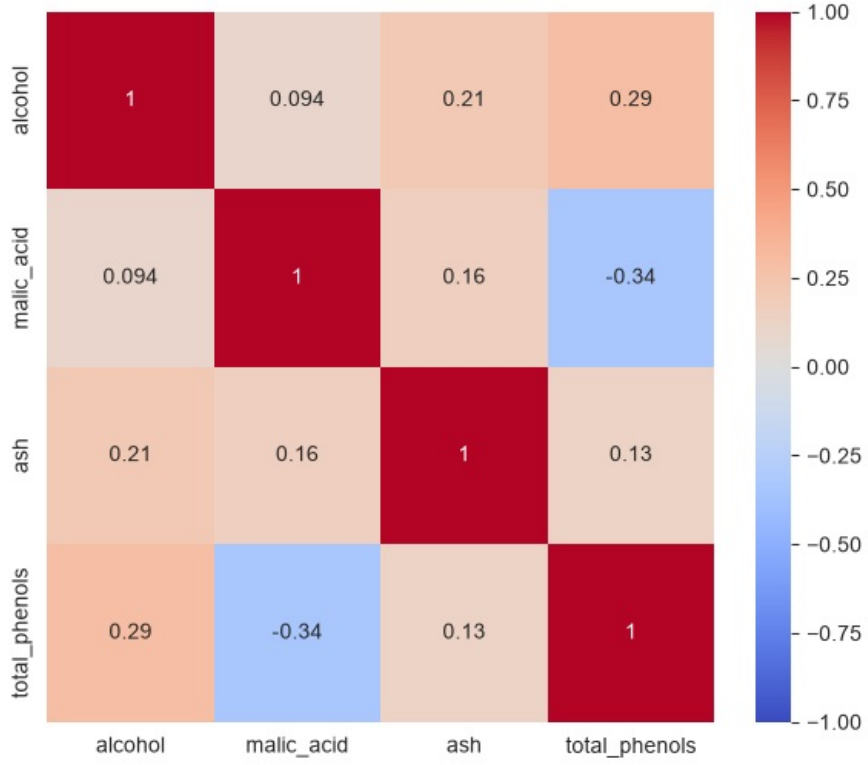
# 8. Comparar con sklearn
Xw_tr, Xw_te, yw_tr, yw_te = train_test_split(
    df_w[vars_w], df_w['Clase_bin'], test_size=0.3,
    random_state=42, stratify=df_w['Clase_bin'])

sc = StandardScaler()
Xw_tr_sc = sc.fit_transform(Xw_tr)
Xw_te_sc = sc.transform(Xw_te)

m_sk = LogisticRegression(max_iter=1000).fit(Xw_tr_sc, yw_tr)
print("Accuracy:", accuracy_score(yw_te, m_sk.predict(Xw_te_sc)))
print("AUC:", roc_auc_score(yw_te, m_sk.predict_proba(Xw_te_sc)[:,1]))
print(classification_report(yw_te, m_sk.predict(Xw_te_sc)))
```

Clase_bin	alcohol		malic_acid		ash		total_phenols	
	mean	std	mean	std	mean	std	mean	std
0	12.632	0.685	2.498	1.249	2.322	0.286	2.025	0.556
1	13.745	0.462	2.011	0.689	2.456	0.227	2.840	0.339





```
                           Logit Regression Results
==============================================================================
Dep. Variable:              Clase_bin   No. Observations:                  178
Model:                          Logit   Df Residuals:                      173
Method:                           MLE   Df Model:                            4
Date:                Thu, 15 Oct 2026   Pseudo R-squ.:                  0.6601
Time:                        11:42:01   Log-Likelihood:                -38.436
converged:                       True   LL-Null:                       -113.07
Covariance Type:            nonrobust   LLR p-value:                 2.929e-31
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const        -50.8074      8.817     -5.763      0.000     -68.087     -33.527
alcohol        3.0418      0.587      5.184      0.000       1.892       4.192
malic_acid    -0.2068      0.297     -0.695      0.487      -0.790       0.376
ash            0.9753      1.034      0.943      0.346      -1.051       3.002
total_phenols  3.3311      0.737      4.521      0.000       1.887       4.775
==============================================================================
ODDS RATIOS

                  Coef       OR  IC 2.5%  IC 97.5%       p
const          -50.8074   0.0000   0.0000    0.0000  0.0000
alcohol          3.0418  20.9433   6.6317   66.1403  0.0000
malic_acid      -0.2068   0.8132   0.4539    1.4568  0.4869
ash              0.9753   2.6518   0.3494   20.1258  0.3456
total_phenols    3.3311  27.9680   6.6002  118.5133  0.0000
Matriz de confusión:

[[110   9]
 [  9  50]]

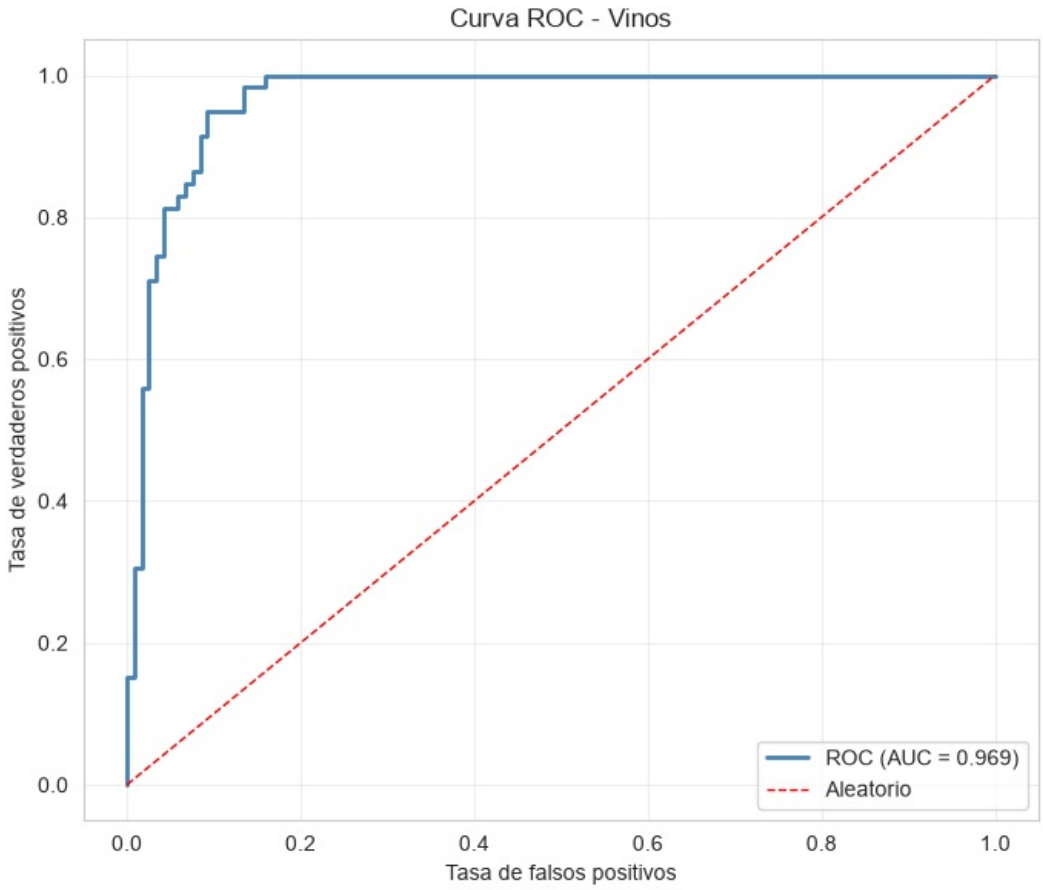
Reporte de clasificación:

              precision    recall  f1-score   support

       resto       0.92      0.92      0.92       119
     clase 0       0.85      0.85      0.85        59

    accuracy                           0.90       178
   macro avg       0.89      0.89      0.89       178
weighted avg       0.90      0.90      0.90       178

Accuracy: 0.8989
```



McFadden R^2: 0.6601
Estadístico HL: 6.8237 | gl: 8 | p-valor: 0.5558

grupo	n	observados	esperados	p_medio	obs_no	esp_no
(-0.000795, 0.00235]	18	0	0.018	0.001	18	17.982
(0.00235, 0.00503]	18	0	0.069	0.004	18	17.931
(0.00503, 0.0119]	18	0	0.154	0.009	18	17.846
(0.0119, 0.0233]	17	0	0.304	0.018	17	16.696
(0.0233, 0.0712]	18	0	0.774	0.043	18	17.226
(0.0712, 0.214]	18	3	2.414	0.134	15	15.586
(0.214, 0.608]	17	8	6.935	0.408	9	10.065
(0.608, 0.865]	18	15	13.542	0.752	3	4.458
(0.865, 0.971]	18	16	16.986	0.944	2	1.014
(0.971, 1.0]	18	17	17.806	0.989	1	0.194

McFadden: 0.6601
Cox-Snell: 0.5677
Nagelkerke: 0.7892
Accuracy: 0.8333333333333334
AUC: 0.9259259259259259

	precision	recall	f1-score	support
0	0.86	0.89	0.88	36
1	0.76	0.72	0.74	18
accuracy			0.83	54
macro avg	0.81	0.81	0.81	54
weighted avg	0.83	0.83	0.83	54

Interpretación de resultados (Vinos: clase 0 vs. resto)

Exploración. Los vinos de clase 0 tienen en promedio más alcohol (13.75 vs. 12.63) y más fenoles totales (2.84 vs. 2.03). Las correlaciones entre predictores son bajas (la mayor es -0.34), así que no hay multicolinealidad preocupante.

Odds Ratios.

- `alcohol` (OR = 20.94, IC 95%: 6.63-66.14, $p < 0.001$): por cada unidad adicional de alcohol, los odds de que el vino sea clase 0 se multiplican por 21, manteniendo lo demás constante.
- `total_phenols` (OR = 27.97, IC 95%: 6.60-118.51, $p < 0.001$): efecto positivo y significativo, con un intervalo muy amplio (estimación poco precisa).
- `malic_acid` (OR = 0.81, $p = 0.487$) y `ash` (OR = 2.65, $p = 0.346$) no son significativas: sus intervalos incluyen el 1.

Evaluación. Accuracy de 0.899 y AUC de 0.969, lo que indica muy buena capacidad de discriminación. La matriz de confusión muestra 9 falsos positivos y 9 falsos negativos de 178 vinos. Hosmer-Lemeshow ($p = 0.556 > 0.05$) no rechaza H_0, por lo que no hay evidencia de falta de ajuste. McFadden = 0.660, Cox-Snell = 0.568 y Nagelkerke = 0.789 indican un ajuste muy bueno.

Comparación con sklearn. En el conjunto de prueba el modelo obtuvo accuracy de 0.833 y AUC de 0.926, menores que en statsmodels (0.899 y 0.969). Es esperable: statsmodels se evaluó con los mismos datos con los que se ajustó, mientras que sklearn se evalúa en datos no vistos, que es una medida más realista. La prueba tiene solo 54 vinos, así que la estimación es variable.

Conclusión. El alcohol y los fenoles totales son los predictores relevantes. Se podría simplificar el modelo quitando `malic_acid` y `ash`.